

AIAA 2205: Introduction to AI Diffusion Model

Li LIU

avrillliu@hkust-gz.edu.cn

The Hong Kong University of Science and Technology (Guangzhou)

Outline

➤ Diffusion Model

What's Generative Models?

A **generative model** learns the **joint probability distribution** $P(x, y)$, where:

- x : the input data (e.g., image, text, sound)
- y : the label or category (e.g., "cat", "dog")

Using this joint distribution, a generative model can:

- **Generate new data samples** x given some label y : $P(x|y)$

A **discriminative model** directly learns the **conditional probability** $P(y|x)$, which means it focuses **only** on how to **separate** or **discriminate** between different classes.

These models **do not attempt to model how the data is generated**. Instead, they just focus on predicting the correct label for given input data.

Aspect	Generative Model	Discriminative Model
Learns	Joint distribution $P(x,y)$	Conditional distribution $P(y x)$
Goal	Understand how data is generated	Predict labels given inputs
Can generate data?	 Yes	 No
Complexity	Often more complex	Often simpler and more direct
Example task	Generate realistic images	Classify images into labels
Sample models	GANs, VAEs, Diffusion Model	Logistic Regression, SVM, BERT

Generative Models

- Variational Autoencoders (VAE) Optimize variational lower bound on likelihood. Useful latent representation, inference queries. But current sample quality not the best.
- Generative Adversarial Networks (GANs) Game-theoretic approach, best samples! But can be tricky and unstable to train, no inference queries.
- Diffusion Model

Recall some mathematics

Gaussian Distribution

1. One-Dimensional Gaussian Distribution

The **Gaussian distribution** (also called the **Normal distribution**) is a continuous probability distribution that is symmetric around its mean. It is widely used in statistics, physics, and engineering due to its mathematical simplicity and the Central Limit Theorem.

Probability Density Function (PDF):

$$f(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

- **Parameters:**

- μ : Mean (center of the distribution).
- σ^2 : Variance (spread of the distribution).
- σ : Standard deviation.

Why Gaussian Distribution is so important?

- **Core Idea of the Central Limit Theorem**

The Central Limit Theorem (CLT) states that the distribution of sample means (or sums) of a large number of independent, identically distributed (i.i.d.) random variables will approximate a Gaussian (Normal) distribution, regardless of the shape of the original population distribution.

Law of Large Numbers is also important!

Python Demonstration of CLT

2. Multivariate Gaussian Distribution

The **Multivariate Gaussian Distribution** generalizes the 1D Gaussian to higher dimensions. It describes the joint distribution of multiple correlated random variables.

Probability Density Function (PDF):

For a d -dimensional random vector $\mathbf{x} = [x_1, x_2, \dots, x_d]^\top$:

$$f(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

- **Parameters:**

- $\boldsymbol{\mu} \in \mathbb{R}^d$: **Mean vector** (center of the distribution).
- $\boldsymbol{\Sigma} \in \mathbb{R}^{d \times d}$: **Covariance matrix** (describes variance and correlation between variables).

Covariance Matrix ($\boldsymbol{\Sigma}$):

- Diagonal elements (Σ_{ii}): Variances of individual variables.
- Off-diagonal elements ($\Sigma_{ij}, i \neq j$): Covariances between variables x_i and x_j .
- $\boldsymbol{\Sigma}$ must be **symmetric** and **positive semi-definite**.

(1) Compute the determinant of the covariance matrix $|\Sigma|$

- The determinant represents the "volume factor" of the covariance matrix, determining the scaling of the distribution.
- **Example:** For a 2x2 covariance matrix:

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{12} & \sigma_2^2 \end{bmatrix}, \quad |\Sigma| = \sigma_1^2 \sigma_2^2 - \sigma_{12}^2$$

(2) Compute the inverse of the covariance matrix Σ^{-1}

- The inverse matrix is used to compute the Mahalanobis distance.
- **Formula** (for 2x2 matrices):

$$\Sigma^{-1} = \frac{1}{|\Sigma|} \begin{bmatrix} \sigma_2^2 & -\sigma_{12} \\ -\sigma_{12} & \sigma_1^2 \end{bmatrix}$$

(3) Compute the Mahalanobis distance

$$D_M = (\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})$$

- The Mahalanobis distance accounts for variable correlations, making it more robust than Euclidean distance.

Assume a 2D Gaussian distribution with parameters:

$$\boldsymbol{\mu} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \boldsymbol{\Sigma} = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$$

Compute the probability density at $\mathbf{x} = [1, 1]^\top$.

Step 1: Compute the determinant

$$|\boldsymbol{\Sigma}| = (1)(1) - (0.5)(0.5) = 0.75$$

Step 2: Compute the inverse matrix

$$\boldsymbol{\Sigma}^{-1} = \frac{1}{0.75} \begin{bmatrix} 1 & -0.5 \\ -0.5 & 1 \end{bmatrix} = \begin{bmatrix} 1.333 & -0.666 \\ -0.666 & 1.333 \end{bmatrix}$$

Step 3: Compute the Mahalanobis distance

$$\mathbf{x} - \boldsymbol{\mu} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$D_M = \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 1.333 & -0.666 \\ -0.666 & 1.333 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = 1.333 - 0.666 - 0.666 + 1.333 = 1.334$$

Step 4: Compute the probability density

$$f(\mathbf{x}) = \frac{1}{(2\pi)^{2/2} \sqrt{0.75}} \exp\left(-\frac{1.334}{2}\right) \approx \frac{1}{5.441} \times 0.513 \approx 0.094$$

Norm of a Matrix

Matrix Norms: An Overview

A **matrix norm** is a scalar value that provides a measure of the "size" or "magnitude" of a matrix. It generalizes the concept of vector norms to matrices and is widely used in linear algebra, numerical analysis, and machine learning. Matrix norms are particularly useful for analyzing the behavior of matrices in various mathematical and computational tasks.

Definition

Formally, a matrix norm is a function $\| \cdot \|$ that maps a matrix A to a non-negative real number, satisfying the following properties for all matrices A and B , and scalars c :

1. **Non-negativity:** $\|A\| \geq 0$, and $\|A\| = 0$ if and only if $A = 0$.
2. **Scalar multiplication:** $\|cA\| = |c| \cdot \|A\|$.
3. **Triangle inequality:** $\|A + B\| \leq \|A\| + \|B\|$.
4. **Submultiplicativity** (for matrix norms): $\|AB\| \leq \|A\| \cdot \|B\|$.

1. Frobenius Norm:

- Definition: $\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2}$, where A is an $m \times n$ matrix.
- Intuition: It measures the Euclidean distance of the matrix when its elements are treated as a vector.
- Applications: Used in machine learning for regularization, dimensionality reduction, and optimization problems.

2. Spectral Norm (Operator Norm):

- Definition: $\|A\|_2 = \sigma_{\max}(A)$, where $\sigma_{\max}(A)$ is the largest singular value of A .
- Intuition: It measures the maximum singular value, which corresponds to the maximum extent to which the matrix can stretch a vector.
- Applications: Important in numerical linear algebra, stability analysis, and deep learning for understanding the impact of weight matrices on gradients.

3. 1-Norm (Column Sum Norm):

- Definition: $\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}|$, which is the maximum absolute column sum.
- Intuition: It measures the maximum absolute sum of elements in any column.
- Applications: Used in optimization and robustness analysis.

4. Infinity-Norm (Row Sum Norm):

- Definition: $\|A\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|$, which is the maximum absolute row sum.
- Intuition: It measures the maximum absolute sum of elements in any row.
- Applications: Useful in error analysis and bounding errors in numerical computations.

5. Nuclear Norm:

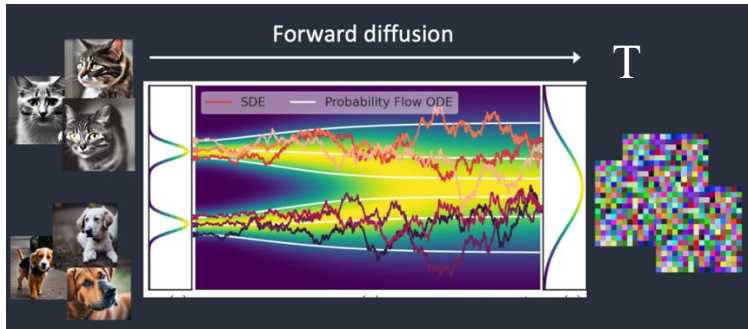
- Definition: $\|A\|_* = \sum_{i=1}^r \sigma_i$, where σ_i are the singular values of A , and r is the rank of A .
- Intuition: It measures the sum of the singular values, encouraging low-rank solutions.
- Applications: Widely used in matrix completion, compressed sensing, and regularization.

Difference between them

Property	Matrix Norm	Determinant
Applies to	Any matrix (square or rectangular)	Only square matrices
Output type	Non-negative real number	Real (or complex) scalar
Meaning	Size / magnitude	Volume scaling / invertibility
Zero value indicates	Only zero matrix	Matrix is singular (non-invertible)
Used in	Analysis, optimization, machine learning	Linear algebra, geometry

Diffusion model

➤ Forward diffusion



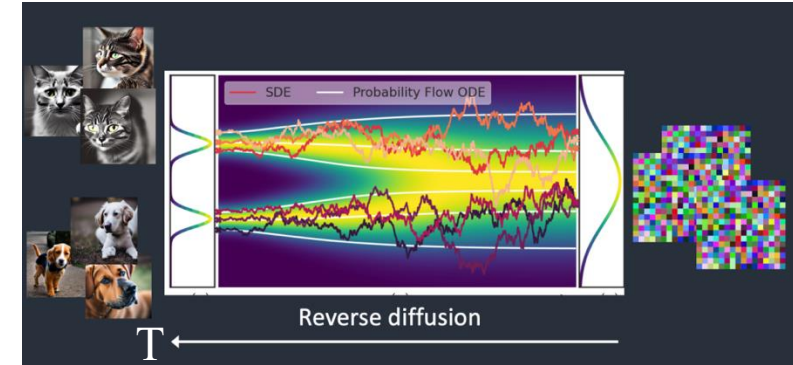
Forward diffusion turns a photo into noise.

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$



Forward diffusion of a cat image.

➤ Reverse diffusion

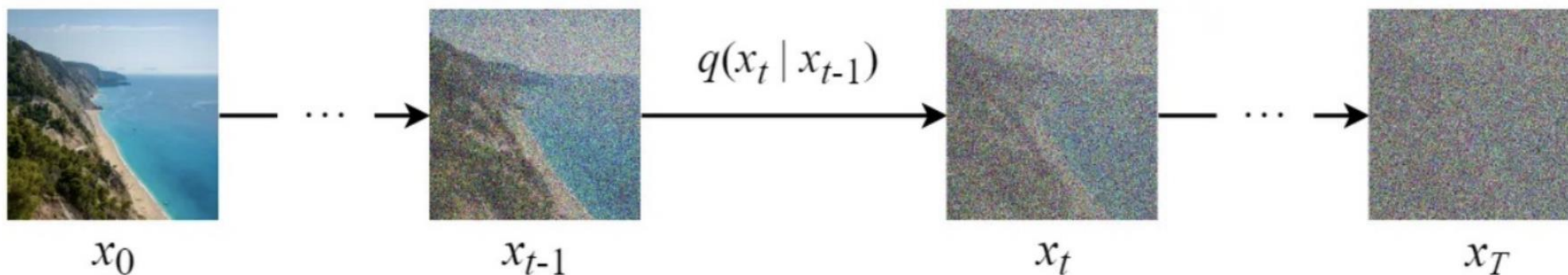


The reverse diffusion process recovers an image.

Ho J, Jain A, Abbeel P. Denoising diffusion probabilistic models. NeurIPS, 2020

Song Y, et al. Score-based generative modeling through stochastic differential equations. arXiv 2020

Forward Diffusion Process



Distribution of the
noised images

Output

Mean μ_t

Variance Σ_t

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

Notations:

t : time step (from 0 to T)

x_0 : a data sampled from the real data distribution $q(x)$ (i.e. $x_0 \sim q(x)$)

β_t : variance schedule ($0 \leq \beta_t \leq 1$, and $\beta_0 = \text{small number}$, $\beta_T = \text{large number}$)

I : identity matrix

Stochastic Transformation

- Recall the reparameterization trick:
 - When $x \sim P(x) = N(x|\mu, \sigma^2)$
then $x = \sigma\epsilon + \mu$ where $\epsilon \sim N(\epsilon|0,1)$
- Since $q(\mathbf{x}_t|\mathbf{x}_{t-1}) = N(\mathbf{x}_t|\sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$
Then $\mathbf{x}_t = \sqrt{1 - \beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\epsilon$ where $\epsilon \sim N(\epsilon|\mathbf{0}, \mathbf{I})$

Stochastic Transformation

- We can speed up the noise process by computing $\mathbf{x}_t$ in one step:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}$$

where $\boldsymbol{\epsilon} \sim N(\boldsymbol{\epsilon} | \mathbf{0}, \mathbf{1})$

$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i \text{ and } \alpha_i = 1 - \beta_i$$

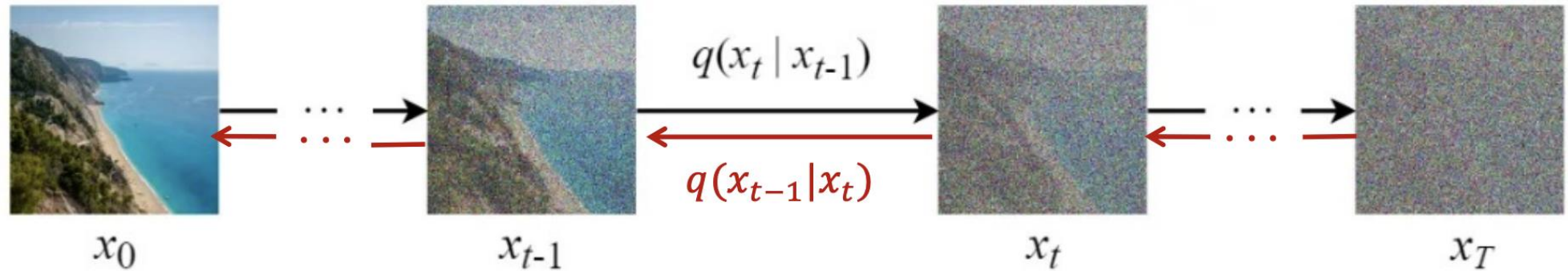
See <https://medium.com/@steinsfu/diffusion-model-clearly-explained-cd331bd41166> for derivation

- In the limit, $\mathbf{x}_\infty$ is a random vector from an isotropic Gaussian

$$\lim_{t \rightarrow \infty} \mathbf{x}_t = \sqrt{\bar{\alpha}_\infty} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_\infty} \boldsymbol{\epsilon} = \boldsymbol{\epsilon} \text{ since } \bar{\alpha}_\infty \rightarrow 0$$

Reverse Denoising Process

From Steins (medium.com)



- Forward factorization: $q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) = q(\mathbf{x}_0) \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1})$
- Reverse factorization: $q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) = \prod_{t=1}^T q(\mathbf{x}_{t-1} | \mathbf{x}_t) q(\mathbf{x}_T)$
 - Since joint distribution is Gaussian then $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ is also Gaussian
 - $q(\mathbf{x}_{t-1} | \mathbf{x}_t) = N(\mathbf{x}_{t-1} | \tilde{\mu}_t(\mathbf{x}_t, t), \sigma_t I)$

Reverse Conditional Gaussian

- The reverse conditional $q(\mathbf{x}_{t-1}|\mathbf{x}_t) = N(\mathbf{x}_{t-1}|\tilde{\mu}_t(\mathbf{x}_t, t), \sigma_t \mathbf{I})$ does not have a closed form, but Ho, Jain and Abbeel (2020) derived the following approximation for $\tilde{\mu}_t$:

$$\tilde{\mu}_t(\mathbf{x}_t, t) \approx \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_t \right)$$

where $\boldsymbol{\epsilon}_t = N(\boldsymbol{\epsilon}|\mathbf{0}, \mathbf{I})$ is the noise introduced at step t

- We do not know $\boldsymbol{\epsilon}_t$, but we can train a neural network $\epsilon_\theta(\mathbf{x}_t, t)$ to approximate it:

$$\text{Minimize } L(\theta) = \|\boldsymbol{\epsilon}_t - \epsilon_\theta(\mathbf{x}_t, t)\|_2^2$$

Training Algorithm

Repeat

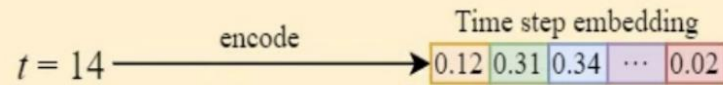
- $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
- $t \sim \text{uniform}(\{1, \dots, T\})$
- $\boldsymbol{\epsilon} \sim N(\mathbf{0}, \mathbf{I})$
- $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}$
- $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta \nabla_{\boldsymbol{\theta}} ||\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)||_2^2$

Until convergence

Training Algorithm

For each training step:

1. Randomly select a time step & encode it



2. Add noise to image



$$x_t = \sqrt{\bar{a}_t} x_0 + \sqrt{1 - \bar{a}_t} \epsilon$$

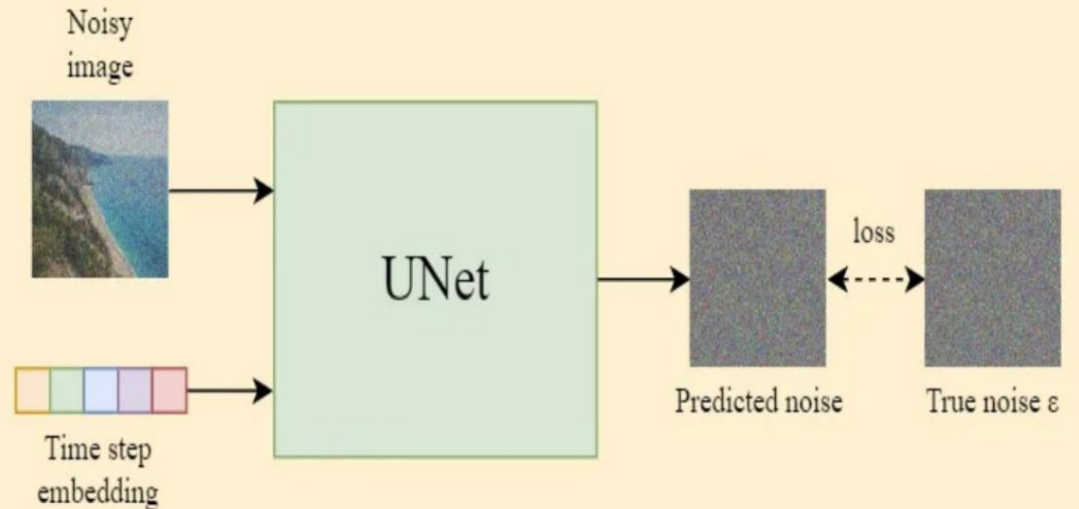
Adjust the amount of noise according to the time step t

$$\epsilon \sim \mathcal{N}(0, 1)$$

$$\alpha_t = 1 - \beta_t$$

$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$

3. Train the UNet



From Steins (medium.com)

Data Generation Algorithm

- $\mathbf{x}_T \sim N(\mathbf{x}|\mathbf{0}, I)$
- For $t = T, \dots, 1$ do
 - $\epsilon \sim N(\epsilon|\mathbf{0}, I)$ if $t > 1$, else $\epsilon = \mathbf{0}$
 - $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sqrt{\sigma_t} \epsilon$
- Return $\mathbf{x}_0$

Data Generation Algorithm

1. Sample a Gaussian noise

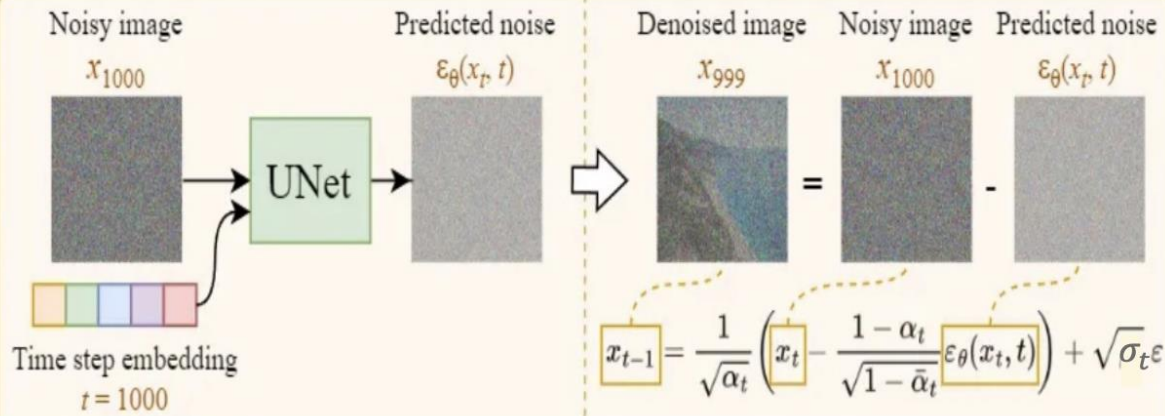
$$x_T \sim N(0, I)$$

E.g. $T = 1000$

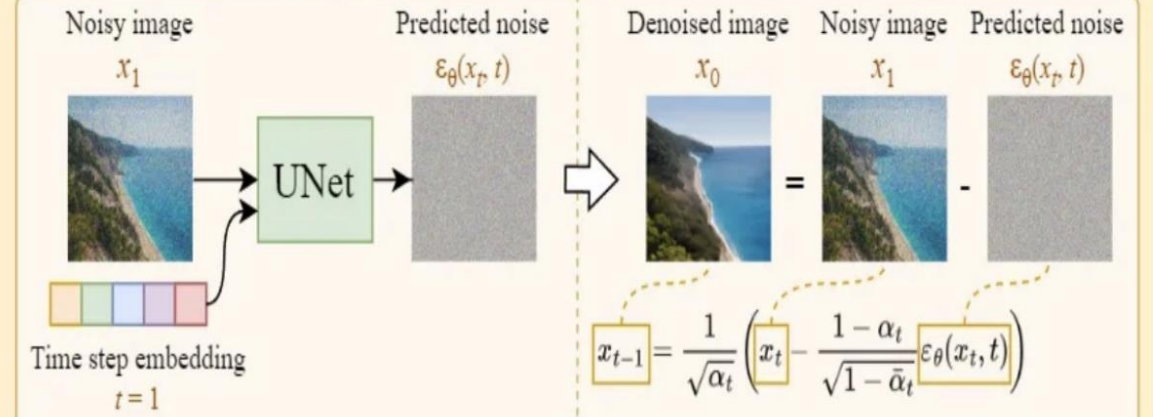
$$x_{1000} \sim N(0, I)$$



2. Iteratively denoise the image



...



3. Output the denoised image

Denoised image x_0



From Steins (medium.com)

Training Algorithm

Repeat

- $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
- $t \sim \text{uniform}(\{1, \dots, T\})$
- $\epsilon \sim N(\mathbf{0}, I)$
- $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$
- $\theta \leftarrow \theta + \eta \nabla_{\theta} ||\epsilon - \epsilon_{\theta}(\mathbf{x}_t, t)||_2^2$

Until convergence

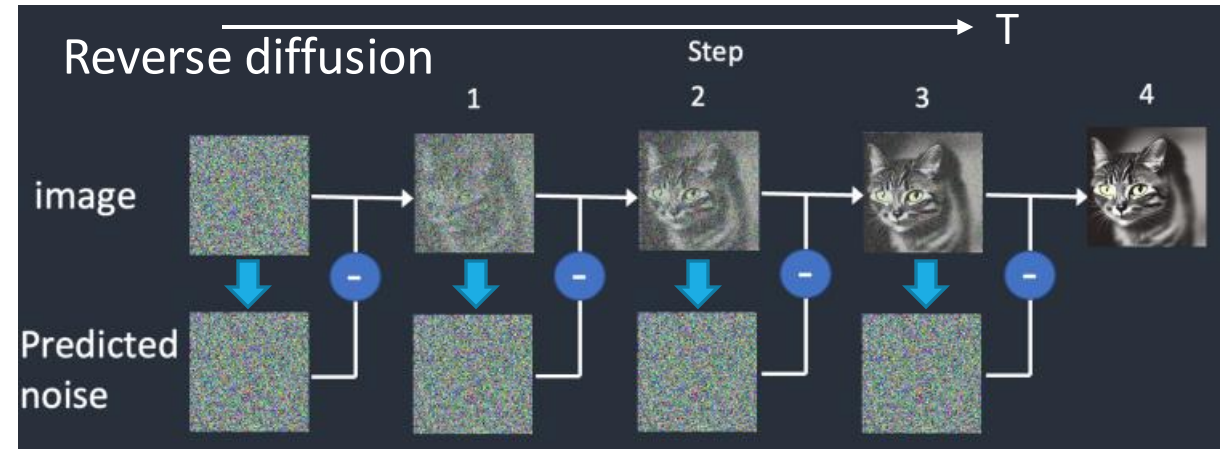
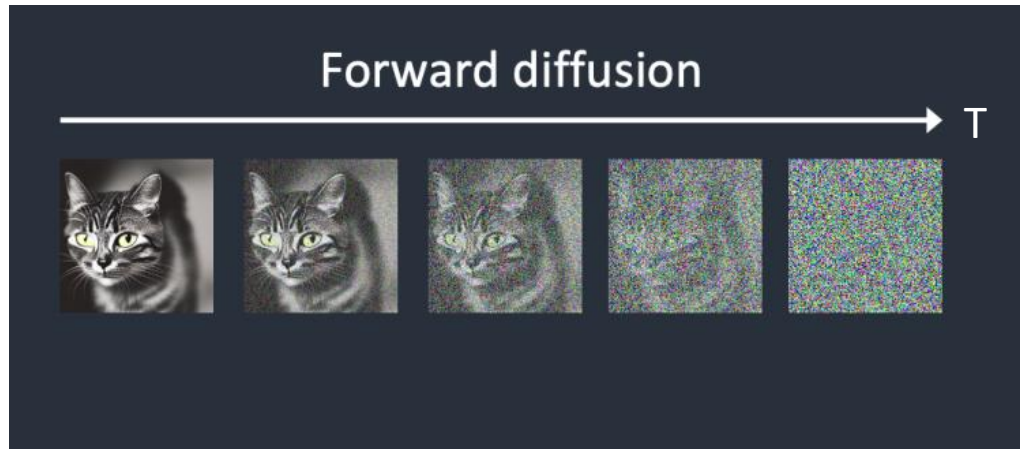
Data Generation Algorithm

- $\mathbf{x}_T \sim N(\mathbf{x}|\mathbf{0}, I)$
- For $t = T, \dots, 1$ do
 - $\epsilon \sim N(\epsilon|\mathbf{0}, I)$ if $t > 1$, else $\epsilon = \mathbf{0}$
 - $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sqrt{\sigma_t} \epsilon$
- Return $\mathbf{x}_0$

Stable diffusion model



- **Motivation:** diffusion models (DMs) typically operate directly in **pixel space**
- Optimization of powerful DMs often consumes hundreds of GPU days (*e.g.*, 150 -1000 V100 days)
- Inference is expensive



Ho J, Jain A, Abbeel P. Denoising diffusion probabilistic models. NeurIPS, 2020 (DDPM)

Saharia C, et al. Photorealistic text-to-image diffusion models with deep language understanding. NeurIPS, 2022 (Imagen)

Ramesh A, et al. Hierarchical text-conditional image generation with clip latents, arXiv 2022 (DALLE-2)

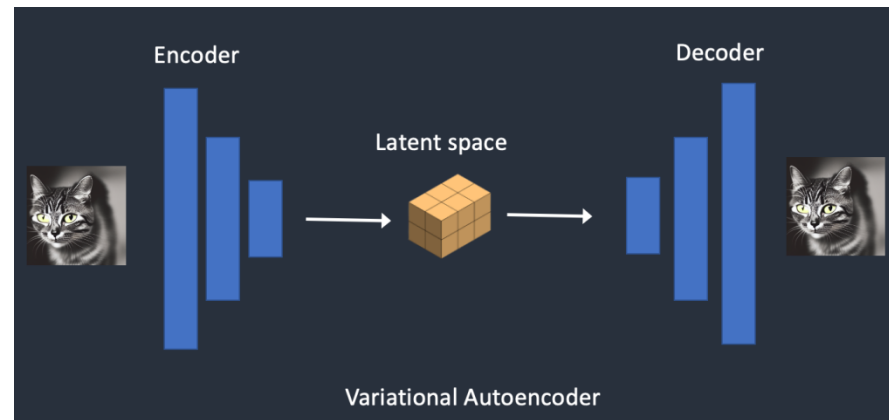
Stable diffusion model

➤ Latent diffusion model

- **Stable Diffusion is a latent diffusion model.** Instead of operating in the high-dimensional image space (*e.g.*, a $512 \times 512 \times 3$ image is a **786,432-dimensional space**), it first compresses the image into the latent space. The **latent space** ($64 \times 64 \times 4$) is 48 times smaller.

➤ Variational Autoencoder

- Variational autoencoder transforms the image to and from the latent space.



Stable diffusion model

➤ Diffusion process

- Instead of corrupting an image with noise, it corrupts the representation of the image in **latent space** with the **latent noise**.

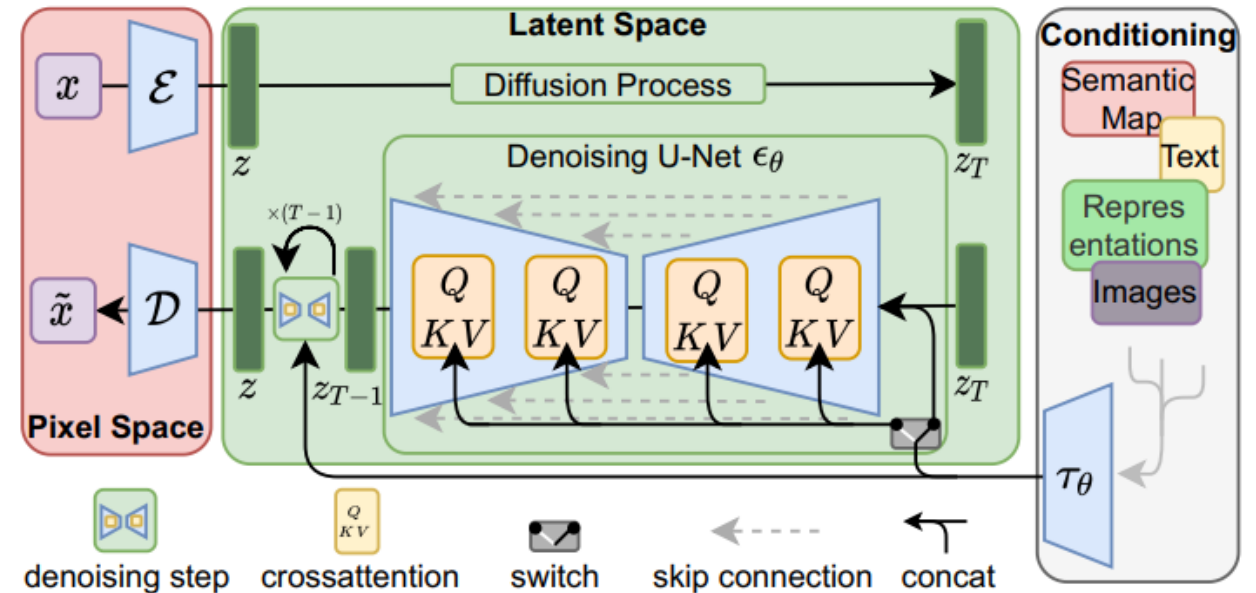
➤ Denoising in latent

➤ Unconditional generation

$$L_{LDM} := \mathbb{E}_{\mathcal{E}(x), \epsilon \sim \mathcal{N}(0,1), t} \left[\|\epsilon - \epsilon_{\theta}(z_t, t)\|_2^2 \right]$$

➤ Conditional generation

$$L_{LDM} := \mathbb{E}_{\mathcal{E}(x), y, \epsilon \sim \mathcal{N}(0,1), t} \left[\|\epsilon - \epsilon_{\theta}(z_t, t, \tau_{\theta}(y))\|_2^2 \right]$$



VAE **encoder**: 2 strided convolutional layers, 3×3 blocks (implemented as ReLU, 3x3 conv, ReLU, 1x1 conv), 256 hidden units; **decoder**: two residual 3×3 blocks, two transposed convolutions [1]

Unet: 860M [2]; ViT-L/14 CLIP text encoder [3]; τ_{θ} : a transformer [4]
T=250

- [1] Van Den Oord A, Vinyals O. Neural discrete representation learning. NIPS, 2017
- [2] Ronneberger O, Fischer P, Brox T. U-net: Convolutional networks for biomedical image segmentation. MICCAI 2015
- [3] A. Radford, et al., Learning transferable visual models from natural language supervision. ICML 2021
- [4] Vaswani, A., Shazeer, N., Parmar, N., et al. Attention is all you need. NIPS 2017

Text-to-image step-by-step

- Give Stable Diffusion a text prompt, and it returns an image.
- **Step 1.** Stable Diffusion generates a random tensor in the latent space.
- **Step 2.** The noise predictor U-Net takes the latent noisy image and text prompt as input and predicts the noise, also in latent space.
- **Step 3.** Subtract the latent noise from the latent image. This becomes your new latent image.

Step 1

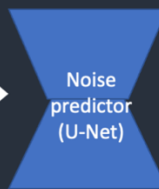


Random tensor in latent space (4x64x64).
Controlled by seed.

Step 2



Image in latent space.



Predicted noise in latent space.

Text Prompt

Step 3

New latent image



=

Latent image



-

Predicted noise



Steps 2 and 3 are repeated for a certain number of sampling steps, for example, 20 times.

Text-to-image step-by-step

- **Give Stable Diffusion a text prompt, and it returns an image.**
- **Step 4.** Finally, the decoder of VAE converts the latent image back to pixel space. This is the image you get after running the Stable Diffusion.

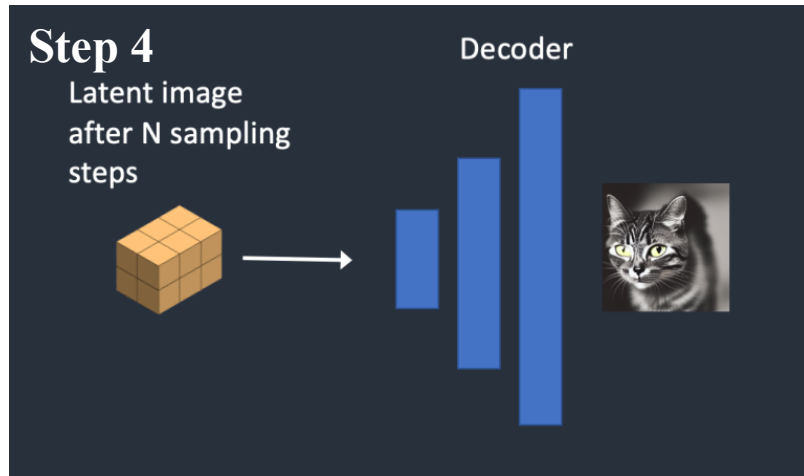


Image at each sampling step.

- Let's see a python code for the unconditional DM and conditional DM.

Thank you!